That's a great idea.

In fact, I would **not** organize it by "Webpack 5 features." Instead, I'd organize it by **what a senior engineer actually needs to know**. Webpack 5 isn't a completely different tool—most of Webpack 2 concepts (Entry, Output, Loaders, Plugins, Code Splitting, etc.) still apply. The biggest differences are new capabilities, simplifications, and best practices.

I'd create a companion cheat sheet titled:

Webpack 5+ Companion Cheat Sheet

Modern Features & Best Practices (Interview + Production)

This would **only** contain topics that are either:

- New in Webpack 5
- Significantly changed from Webpack 2/3/4
- Frequently asked in modern interviews
- Useful in real-world production applications

Suggested Contents

Part 1 — Modern Asset Management

Topic	Why it Matters
Asset Modules	Replacement for `file-loader`, `url-loader`, and `raw-loader`
`asset/resource`	Images, fonts
`asset/inline`	Base64 assets
`asset/source`	Import raw source
`asset`	Automatic selection
Asset Generator options	Output filenames
Asset Parser options	Inline thresholds
SVG handling	Best practices

Topic	Why it Matters
Fonts	Modern configuration

Part 2 — Modern Performance

Topic
Persistent Filesystem Cache
Deterministic Module IDs
Improved Long-term Caching
Runtime Chunk
SplitChunks improvements
Better Tree Shaking
Side Effects
Nested Tree Shaking
Package Exports
Build Performance

Part 3 — Module Federation

One of the biggest Webpack 5 features.

Topics:

- Why Module Federation?
- Host
- Remote
- Shared libraries
- Version negotiation
- Singleton packages
- Micro Frontends
- Production architecture
- Interview questions

Part 4 — CSS Improvements

- CSS Modules
- PostCSS
- Autoprefixer
- Sass
- Less
- CSS Optimization
- CSS Minification
- CSS Code Splitting

Part 5 — Modern JavaScript

- Top-level `await`
- `experiments`
- Web Workers
- WebAssembly
- Dynamic `import()`
- Import Assertions (overview)
- Modern ESM support

Part 6 — Production Configuration

- `webpack-merge`
- Multiple configs
- Environment variables
- DefinePlugin
- dotenv
- Dev vs Prod
- Source Map strategy
- Performance hints

Part 7 — Build Optimization

- Bundle Analyzer
- Duplicate package detection

- Speed Measurement
- Parallel builds
- Cache invalidation
- Build profiling

Part 8 — Modern Best Practices

- Monorepo support
- Package exports
- Path aliases
- Externals
- CDN strategy
- Vendor splitting
- Lazy loading
- Route-based splitting
- Prefetch vs Preload
- Compression (Gzip/Brotli)

Part 9 — Security

- Secrets
- Environment variables
- CSP
- SRI
- Dependency auditing
- Supply-chain security

Part 10 — Frequently Asked Interview Questions

Approximately **40–50 modern Webpack interview questions**, such as:

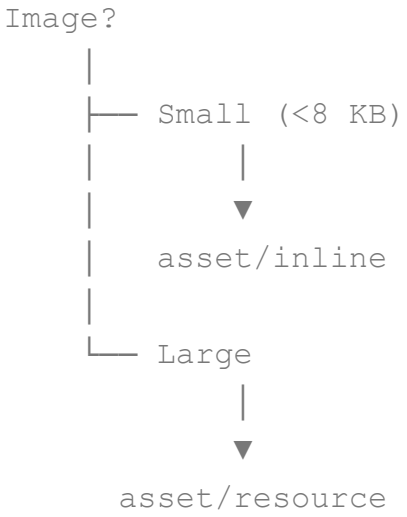
- Why was `file-loader` removed?
- Explain Asset Modules.
- Module Federation vs npm package.
- Runtime Chunk.
- Deterministic Module IDs.
- Filesystem Cache.
- Side Effects.

- Tree Shaking internals.
- SplitChunks strategies.
- Prefetch vs Preload.
- Webpack vs Vite.
- Webpack vs Turbopack.
- When would you still choose Webpack?

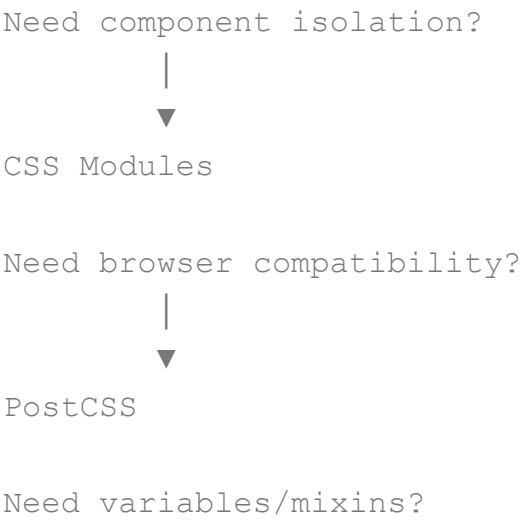
Modern Decision Trees

For example:

Which Asset Type?



CSS Decision



```
        |
        ▼
Sass
```

Caching Decision

```
Long-term browser caching?
        |
        ▼
contenthash

Need faster rebuilds?
        |
        ▼
Filesystem Cache
```

Size

Unlike the previous four-part guide, this would be much shorter because it only covers **new or changed** concepts.

Approximately:

- **20–30 pages**
- **150–200 quick-reference entries**
- Around **150 code snippets**
- Focused entirely on **Webpack 5+**

I also recommend adding one final section

"Webpack 2 → Webpack 5 Migration Cheat Sheet"

This would be extremely valuable because your primary course targets Webpack 2. It would include a concise mapping like:

Webpack 2	Webpack 5
`file-loader`	`asset/resource`

Webpack 2	Webpack 5
`url-loader`	`asset/inline` / `asset`
`raw-loader`	`asset/source`
`extract-text-webpack-plugin`	`MiniCssExtractPlugin`
`System.import()`	`import()`
`CommonsChunkPlugin`	`SplitChunksPlugin`
`CleanWebpackPlugin` (common)	`output.clean: true` (often sufficient)
`optimization.namedModules`	Automatic deterministic module IDs
Manual caching strategies	Persistent filesystem cache

I think this migration section would be especially useful because it bridges the gap between your existing course notes and modern Webpack practices, making it much easier to translate what you've learned into current production projects and interview discussions.